

Project Notes — Tic Tac Toe

DOM Manipulation with HTML, CSS & JavaScript

Project Structure

Three files work together: tic.html (structure), tac.css (styling), toe.js (game logic). The HTML links the CSS with `<link>` and the JS with a `<script>` tag at the end of the body.

```
<link rel="stylesheet" href="tac.css" /> <!-- in <head> -->
<script src="app.js"></script> <!-- end of <body> -->
```

Note: the script tag references "app.js", but your file is named toe.js — rename the file to app.js, or change the src to "toe.js", so the browser actually loads the game logic.

HTML — Page Structure

Key elements the JavaScript will hook into:

Element	Purpose
.box (x9)	The 9 clickable squares of the grid, as buttons
.game	Wraps the 9 boxes in a flex-wrap grid
#reset-btn	Resets the board mid-game
.msg-container	Hidden overlay shown at game end (win/draw)
#msg	Text inside the overlay ("Winner" / draw message)
#new-btn	Starts a new game from the end-game overlay

The .msg-container starts with class="hide" so it's invisible until the game ends.

CSS — Layout Highlights

.container and .game use Flexbox to center the 3x3 grid on the page:

```
.game {
  height: 60vmin;
  width: 60vmin;
  display: flex;
  flex-wrap: wrap; /* lets 9 boxes wrap into 3 rows */
  justify-content: center;
  align-items: center;
  gap: 1.5vmin;
}
```

vmin is a unit relative to the smaller of the viewport's width/height — it keeps the board square and responsive on any screen size.

.hide toggles visibility of the end-game overlay:

```
.hide {  
  display: none;  
}
```

JavaScript — Selecting Elements

querySelectorAll() grabs every element matching a class (returns a list-like NodeList). querySelector() grabs the first element matching a selector (here, IDs, which are unique).

```
let boxes = document.querySelectorAll(".box"); // NodeList of all 9 boxes  
let resetBtn = document.querySelector("#reset-btn");  
let newGameBtn = document.querySelector("#new-btn");  
let msgContainer = document.querySelector(".msg-container");  
let msg = document.querySelector("#msg");
```

Tracking Game State

```
let turn0 = true; // true = 0's turn, false = X's turn  
let count = 0; // counts moves made, to detect a draw at 9
```

Win Patterns

The 9 boxes are indexed 0-8 like this:

```
0 | 1 | 2  
3 | 4 | 5  
6 | 7 | 8
```

Each possible winning line (3 rows, 3 columns, 2 diagonals) is stored as a triplet of indexes:

```
const winPatterns = [  
  [0, 1, 2], [0, 3, 6], [0, 4, 8],  
  [1, 4, 7], [2, 5, 8], [2, 4, 6],  
  [3, 4, 5], [6, 7, 8],  
];
```

Handling a Box Click

forEach attaches a click listener to every box. On click: write the current player's symbol, flip turn0 for the next turn, disable that box so it can't be clicked again, increment count, then check for a winner or draw.

```
boxes.forEach((box) => {  
  box.addEventListener("click", () => {  
    if (turn0) {  
      box.innerText = "O";  
      turn0 = false;  
    }  })  
})
```

```

    } else {
      box.innerText = "X";
      turn0 = true;
    }
    box.disabled = true;
    count++;

    let isWinner = checkWinner();

    if (count === 9 && !isWinner) {
      gameDraw();
    }
  });
});

```

Note: `checkWinner()` runs after every single move, not just at move 9 — so a win is detected the moment it happens, while the draw check (`count === 9`) only matters once the board is full.

Checking for a Winner

Loops through every pattern in `winPatterns`. For each one, reads the text in the 3 related boxes. If all three are filled (not empty) and all three match each other, that player has won.

```

const checkWinner = () => {
  for (let pattern of winPatterns) {
    let pos1Val = boxes[pattern[0]].innerText;
    let pos2Val = boxes[pattern[1]].innerText;
    let pos3Val = boxes[pattern[2]].innerText;

    if (pos1Val !== "" && pos2Val !== "" && pos3Val !== "") {
      if (pos1Val === pos2Val && pos2Val === pos3Val) {
        showWinner(pos1Val);
        return true;
      }
    }
  }
};

```

Showing a Winner, a Draw, and Resetting

`showWinner()` — uses a template literal to insert the winning symbol into the message:

```

const showWinner = (winner) => {
  msg.innerText = `Congratulations, Winner is ${winner}`;
  msgContainer.classList.remove("hide"); // reveal the overlay
  disableBoxes();
};

```

`gameDraw()` — same idea, for when all 9 boxes fill with no winner:

```
const gameDraw = () => {
  msg.innerText = `Game was a Draw.`;
  msgContainer.classList.remove("hide");
  disableBoxes();
};
```

disableBoxes() / enableBoxes() — loop over every box to lock or unlock + clear it:

```
const disableBoxes = () => {
  for (let box of boxes) {
    box.disabled = true;
  }
};

const enableBoxes = () => {
  for (let box of boxes) {
    box.disabled = false;
    box.innerText = "";
  }
};
```

resetGame() — puts the whole board back to a fresh starting state:

```
const resetGame = () => {
  turn0 = true;
  count = 0;
  enableBoxes();
  msgContainer.classList.add("hide"); // hide the overlay again
};
```

Wiring the Buttons

Both the "Reset Game" button and the "New Game" button (in the overlay) call resetGame():

```
newGameBtn.addEventListener("click", resetGame);
resetBtn.addEventListener("click", resetGame);
```

Core Concepts Used in This Project

Concept	Where it's used
querySelector / querySelectorAll	Selecting DOM elements to read or change
addEventListener("click", fn)	Running code in response to a button click
classList.add() / remove()	Toggling the .hide class to show/hide the overlay
Template literals (`\${}`)	Building the winner/draw message text
Array of arrays	winPatterns stores all 8 possible winning lines
for...of / forEach	Looping over the box NodeList

Arrow functions	All the game-logic functions (resetGame, checkWinner, etc.)
.disabled property	Locking a box so it can't be clicked again after a move